A PRACTICAL BLUEPRINT FOR BUILDING THE COMPLETE PRODUCT

SonicScope AI

Music intelligence from MP3 audio: genre recognition, spectrogram-based deep learning, acoustic analysis and an interactive dashboard.

Implementation plan | Datasets, downloads, setup, ML pipeline, API, frontend, evaluation, QA and delivery

CORE TRAINING DATA	ORIGINAL AI WORK	FINAL PRODUCT
FMA Small: 8,000 clips 8 genres / 30 seconds	Random Forest baseline + log-mel CNN	MP3 upload + analysis dashboard + similarity

WHAT YOU ACTUALLY DOWNLOAD For the main project, download **FMA Small** and **FMA metadata**. The giant MTG-Jamendo audio archive is **not required**. Optional pretrained model files are linked separately.

Project category: AI & Data Science mini-project | Version 1.0 | 17 October 2026



START HERE / DOWNLOADS

Download these first

Create a dedicated project folder, then download exactly these two files. You do not need the large MTG-Jamendo audio collection for this mini-project.

Required item	Download size	Save into
FMA Small (fma_small.zip) 8,000 tracks; 30-second audio; 8 genres	7.2 GiB	data/downloads/fma_small.zip
FMA metadata (fma_metadata.zip) Track labels, split assignments, metadata	342 MiB	data/downloads/fma_metadata.zip

1. Download FMA Small - official direct ZIP https://os.unil.cloud.switch.ch/fma/fma_small.zip
2. Download FMA metadata - official direct ZIP https://os.unil.cloud.switch.ch/fma/fma_metadata.zip
Official FMA publisher instructions and checksums https://github.com/mdeff/fma

Exact folder layout for the first download

```
SonicScopeAI/
  data/
    downloads/
      fma_small.zip
      fma_metadata.zip
    raw/                  # extract here later
  models/
    pretrained/           # optional AI model files later
```

Download only after the core model works

For optional mood and instruments, fetch one shared pretrained Discogs-EffNet encoder and two classifier heads, including their matching JSON labels. Chapter 10 contains six direct model-file links. Do not download MTG-Jamendo full-size audio or DEAM for the v1 build.

> **STORAGE AND SAFETY** Plan for approximately 25-35 GiB free space. Verify both publisher SHA-1 checksums before extracting them. Detailed PowerShell and Linux/macOS commands start in Chapter 4.

Fastest implementation sequence

Downloads → metadata manifest → spectrograms → Random Forest → CNN → FastAPI → Next.js → evaluation → optional tags.

How to use this guide

Proceed in sequence and complete the verification gates before adding optional features. Links and commands are included where they are needed. The numbered headings in the PDF are also easy to find using search.

> **DEFINITION OF DONE** Upload a fresh MP3 from a browser and produce actual signal features, visualizations and a prediction from **your trained genre CNN**. Show held-out evaluation metrics, not invented example accuracy.

Implementation index

01 The product and academic scope

02 Exactly what to download

03 Install software and create the workspace

04 Download, verify and extract the datasets

05 Repository structure and responsibilities

06 Build the dataset manifest correctly

07 Preprocess audio and generate visual data

08 Train the classical baseline

09 Train your original 2D CNN

10 Optional mood and instrument inference

11 Numerical music analysis

12 Song similarity engine

13 Backend: FastAPI endpoint contract

14 Frontend product specification

15 Execution flow and integration order

16 Evaluation protocol and experiments

17 Security, reliability and tests

18 14-day implementation schedule

19 Academic report and deliverables

20 Risks, fallbacks and non-goals

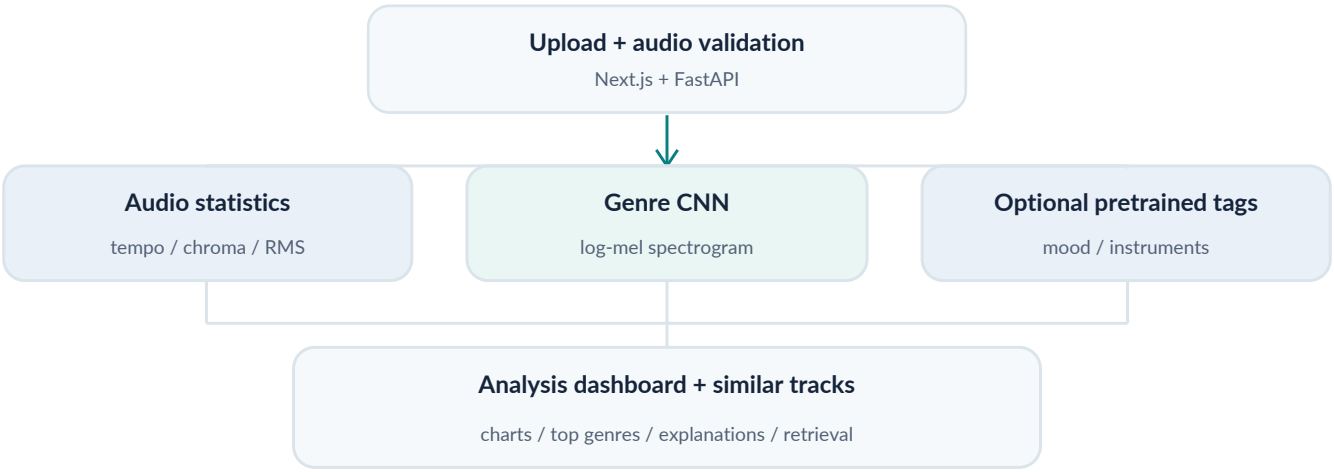
21 Primary references and direct links

22 Final execution checklist

Decision	Recommended
Scope	Single-user local or lab-deployed application. No accounts, streaming or music generation.
Primary ML goal	Eight-way genre recognition on FMA Small with an independently trained 2D CNN.
Validation	Use official FMA train / validation / test assignments at song level.
Optional tags	Essentia pretrained music models only after the standalone demo works.
Timebox	14 working days as a baseline; allow extra time for dataset downloads and training.

01 The product and academic scope

SonicScope AI is a web application for extracting understandable musical intelligence from audio. Its unique mini-project contribution is the comparison between feature-engineered machine learning and image-like spectrogram deep learning.



User journey

- Open the dashboard and upload a valid .mp3 file.
- Preview audio and view file metadata and upload status.
- Backend validates and decodes the audio, then computes audio features and a log-mel spectrogram.
- Your CNN predicts eight FMA genre classes; the browser displays ranked class scores.
- Charts show waveform, spectrogram, chromagram, beat markers and energy over time.
- An optional pretrained model provides mood and instrument tags; a similarity index finds related FMA clips.

In scope versus out of scope

MVP (must work)	Extensions (only if time remains)
MP3 upload and playback; genre CNN; audio feature extraction; interactive charts	Mood/instrument tags with pretrained model; top-5 similar tracks
Random Forest baseline; evaluation metrics; local repeatable demo	Batch multi-song comparison; PDF export; per-segment predictions
Safe file handling and useful errors	No lyrics transcription, music generation, exact song identification, accounts or streaming

02 Exactly what to download

This is the definitive acquisition list. Download the mandatory two FMA ZIPs before writing any modeling code. Links in this document are clickable.

Priority / artifact	Source + size	Save location
REQUIRED fma_small.zip	FMA Small; 8,000 30-second MP3 tracks; **7.2 GiB** [S1]	data/downloads/fma_small.zip

Priority / artifact	Source + size	Save location
REQUIRED fma_metadata.zip	FMA metadata and features; **342 MiB** [S1]	data/downloads/fma_metadata.zip
OPTIONAL FMA GitHub code	Docs, examples and reference loaders [S1]	references/fma/
OPTIONAL 3 pretrained .pb + 3 .json	Discogs-EffNet encoder and two classification heads [S2]	models/pretrained/
NOT REQUIRED MTG-Jamendo raw audio	Over 55k tracks; even the low quality 30s collection is massive [S3]	Do not download for v1
NOT REQUIRED DEAM	Extra emotion-specific training dataset	Leave for future research

FMA Small direct ZIP https://os.unil.cloud.switch.ch/fma/fma_small.zip
FMA metadata direct ZIP https://os.unil.cloud.switch.ch/fma/fma_metadata.zip
Official FMA overview, checksums and licenses https://github.com/mdeff/fma

> **DISK SPACE** Reserve roughly **25-35 GiB** free for ZIPs, extracted MP3s, spectrogram cache, artifacts and temporary processing. This is a planning allowance, not an exact published dataset size.

03 Install software and create the workspace

Minimum software

Item	Suggested version / guidance
OS	Windows 10/11 for core training/frontend; WSL2 Ubuntu if trying TensorFlow-enabled Essentia
Python	Python 3.11, virtual environment, pip
Node.js	Node 22 LTS or another version supported by the installed Next.js release; npm
Git and editor	Git, VS Code or another preferred IDE
Audio tools	FFmpeg on PATH. Confirm with ffmpeg -version and ffprobe -version.
Hardware	16 GiB RAM sufficient for a cautious pipeline; GPU useful but not mandatory

Windows PowerShell: first commands

```
mkdir SonicScopeAI
cd SonicScopeAI
git init
mkdir data, models, reports, scripts, backend, references
mkdir data/downloads, data/raw, data/processed, data/cache
mkdir models/checkpoints, models/pretrained, models/baseline
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

If PowerShell blocks virtual-environment activation, use the venv Python executable directly or adjust execution policy according to your machine policy. Do not disable system security controls just to run the project.

Python dependencies (initial install)

```
python -m pip install numpy pandas scipy librosa soundfile
python -m pip install scikit-learn torch torchaudio
python -m pip install matplotlib plotly joblib pydantic-settings
python -m pip install pytest httpx
python -m pip install "fastapi[standard]" python-multipart
```

After a successful environment test, record exact dependency versions in a lock or requirements file. Install FFmpeg separately through an official package manager or release and verify it is available on PATH.

04 Download, verify and extract the datasets

Option A: PowerShell browser-safe download

```
Invoke-WebRequest -Uri `
 "https://os.unil.cloud.switch.ch/fma/fma_small.zip" `
 -OutFile "data/downloads/fma_small.zip"
Invoke-WebRequest -Uri `
 "https://os.unil.cloud.switch.ch/fma/fma_metadata.zip" `
 -OutFile "data/downloads/fma_metadata.zip"
```

Large downloads may be more reliable using a resume-capable browser/download manager. If an archive download is interrupted, restart or use a resume-capable client before attempting extraction.

Check publisher SHA-1 digests before extraction

```
(Get-FileHash .\data\downloads\fma_small.zip -Algorithm SHA1).Hash
# Expected: ade154f733639d52e35e32f5593efe5be76c6d70
(Get-FileHash .\data\downloads\fma_metadata.zip -Algorithm SHA1).Hash
# Expected: f0df49ffe5f2a6008d7dc83c6915b31835dfe733
```

Hashes are from the official FMA README [S1]. Compare case-insensitively. If they do not match, do not trust the archive. SHA-1 is used here only for matching the publisher-provided download checksum.

Extract into predictable paths

```
Expand-Archive data/downloads/fma_small.zip data/raw -Force
Expand-Archive data/downloads/fma_metadata.zip data/raw -Force
# Expected: data/raw/fma_small/000/000002.mp3 (example)
# Expected: data/raw/fma_metadata/tracks.csv
```

Option B: Linux / WSL / macOS

```
mkdir -p data/downloads data/raw
BASE=https://os.unil.cloud.switch.ch/fma
curl -L -C - -o data/downloads/fma_small.zip "$BASE/fma_small.zip"
curl -L -C - -o data/downloads/fma_metadata.zip "$BASE/fma_metadata.zip"
cd data/downloads
sha1sum fma_small.zip fma_metadata.zip
cd ../..
unzip -q data/downloads/fma_small.zip -d data/raw
unzip -q data/downloads/fma_metadata.zip -d data/raw
```

DATASET QUALITY GATE Confirm 8,000 expected audio files, CSV availability and successful decoding of at least five randomly chosen MP3s. Some FMA releases have known errata; log unreadable files and exclude them consistently rather than inventing labels.

05 Repository structure and responsibilities

```
SonicScopeAI/
  backend/
    app/
      main.py                # FastAPI entry point
      routes/analyze.py      # upload + prediction API
      services/audio.py      # decode/segment/visuals
      services/genre.py      # CNN inference
      services/tags.py       # optional pretrained tags
      services/similarity.py  # nearest-neighbor retrieval
      schemas.py             # typed response contracts
  web/                      # Next.js application
  scripts/
    01_build_manifest.py
    02_extract_features.py
    03_cache_spectrograms.py
    04_train_baseline.py
    05_train_cnn.py
    06_evaluate.py
    07_build_similarity_index.py
    08_smoke_test.py
  data/
    downloads/ raw/ processed/ cache/
  models/
    checkpoints/ baseline/ pretrained/
  reports/
    figures/ metrics/ screenshots/
  tests/
  README.md  .gitignore  .env.example
```

Version control rules

- Commit source code, settings templates, plots of aggregate metrics and tiny permitted test fixtures.
- Do not commit downloaded datasets, user audio, temporary uploads, .venv, model weights or large spectrogram caches. Put those paths in .gitignore.
- Document dataset and model download sources so another evaluator can reproduce the experiment without sharing copyrighted audio in the repository.

Required generated artifacts

Artifact	Who creates it / purpose
data/processed/manifest.csv	Training script: one row per track with track_id, path, genre, split, optional artist_id
data/processed/features.parquet	Feature extraction: reusable baseline and similarity inputs
data/cache/logmels/	Deterministic 10-second log-mel arrays (.npy), optional disk cache
models/checkpoints/genre_cnn.pt	Best validation-selected CNN checkpoint
models/baseline/random_forest.joblib	Fitted baseline pipeline with preprocessing
reports/metrics/test_metrics.json	Final held-out evaluation results and version metadata
data/processed/similarity_index.joblib	Indexed song features and standardized retrieval vectors

06 Build the dataset manifest correctly

Read the official metadata

The FMA metadata uses a multi-row column header. Read tracks.csv with pandas and select the **small** subset only, rather than guessing genre labels from directory names.

```
import pandas as pd
tracks = pd.read_csv(
    "data/raw/fma_metadata/tracks.csv",
    index_col=0,
    header=[0, 1]
)
small = tracks[tracks[("set", "subset")] == "small"]
for track_id, row in small.iterrows():
    split = row[("set", "split")]
    genre = row[("track", "genre_top")]
    mp3 = f"data/raw/fma_small/{track_id//1000:03d}/{track_id:06d}.mp3"
    # Validate mp3, collect artist ID, save manifest row.
```

Required validation rules

- Only one manifest row per track_id. Track IDs cannot repeat across splits.
- Use the official set/split values **training, validation, test**; do not randomly re-split FMA Small after reading those assignments.
- Use the exact genre_top labels from tracks.csv. Save a fixed alphabetical label-to-index dictionary inside the model checkpoint.
- If artist IDs exist, report any artist overlap across splits as a limitation. Do not silently claim artist-disjoint evaluation.
- Missing or unreadable files are logged to reports/metrics/data_exclusions.csv with the exact reason.

Output manifest schema

Column	Type / example
track_id	integer / 2
path	relative path / data/raw/fma_small/000/000002.mp3
genre	string / Electronic
split	string / training | validation | test
artist_id	optional numeric FMA metadata ID

LEAKAGE PREVENTION **Split at song level first.** Only then divide clips into smaller windows. Never put different 10-second windows of the same track into both train and test sets.

07 Preprocess audio and generate the visual dataset

Deterministic audio processing contract

Property	Implementation default
Audio decode	Convert MP3 to mono PCM; float32 normalized waveform
Sample rate	22,050 Hz for custom CNN and handcrafted features
Duration	Use up to 30 seconds per original training file
Segments	Three non-overlapping 10-second windows per training track
Spectrogram	128 mel bands; n_fft=2048; hop_length=512
Transformation	power-to-dB log mel, normalize with train-only statistics
Tensor	[1, 128, ~431] per 10-second segment, precise frame count from implementation
Cache	Save float16 or compressed arrays on disk; load batches, do not keep whole dataset in RAM

Spectrogram generation: reference algorithm

```
import librosa
import numpy as np

y, sr = librosa.load(mp3_path, sr=22050, mono=True,
                     duration=30)
# Pad short audio if needed; segment into 10-second windows.
mel = librosa.feature.melspectrogram(
    y=segment, sr=sr, n_fft=2048,
    hop_length=512, n_mels=128, power=2.0)
logmel = librosa.power_to_db(mel, ref=np.max)
# Fit global normalization statistics on training windows only.
# Apply those saved values to validation, test and uploads.
```

Exploratory data analysis to include in the report

- Genre-class counts by official split; verify balanced representation.
- Example mel-spectrograms from two to three songs per genre, correctly captioned.
- Average track duration, missing/corrupt sample count and audio decoding summary.
- Distribution of spectral centroid, RMS energy and tempo by genre.

The spectrogram arrays are the project's **visual training dataset**. They are derived from labeled audio; no separate external image dataset is necessary. Use librosa display utilities only for visualization, not as a replacement for model inputs. [S4]

08 Train the classical baseline

Purpose

A Random Forest demonstrates what conventional feature-engineered ML can do, enabling a defensible comparison with the spectrogram CNN. The baseline is not an extra UI feature; it is part of the scientific evaluation.

Suggested feature vector per track

Group	Features
Timbral	MFCC 1-13 mean and std (26 columns)
Tonal	12-bin chroma mean (12 columns)
Spectral	centroid, rolloff, bandwidth, zero-crossing rate: mean and std
Dynamics	RMS energy mean and std
Rhythm	tempo estimate, optional onset-strength summary

Training procedure

- Extract each track vector and cache to features.parquet. Handle NaN/inf and silent clips explicitly.
- Fit a scikit-learn Pipeline: SimpleImputer then RandomForestClassifier. Tuning remains restricted to validation data.
- Choose a small documented parameter grid (e.g., estimators 200/400; max depth 12/None), with one fixed random seed.
- Calculate validation macro F1; save the best model and a JSON metrics summary.
- Test exactly once after final configuration, then save confusion matrix and class-level precision/recall/F1.

Important modeling note

> **FAIR COMPARISON** The CNN and baseline must use the same held-out track IDs, the same eight labels, and track-level metrics. You may tune each model separately on validation but do not choose a winning model using test scores.

09 Train your original 2D CNN

Lightweight architecture specification

Stage	Shape / operation
Input	1 x 128 x ~431 normalized log-mel
Block 1	Conv2d(1, 16, 3, padding=1) + BatchNorm + ReLU + MaxPool2d(2)
Block 2	Conv2d(16, 32, 3, padding=1) + BatchNorm + ReLU + MaxPool2d(2)
Block 3	Conv2d(32, 64, 3, padding=1) + BatchNorm + ReLU + MaxPool2d(2)
Head	AdaptiveAvgPool2d(1) + flatten + Dropout(0.3) + Linear(64, 8)
Output	Eight raw logits; apply softmax for single-label UI scores

Training recipe

Setting	Starting point
Loss / optimizer	CrossEntropyLoss / AdamW
Learning rate / batch	1e-3 / 16 or 32 (reduce if GPU memory is insufficient)
Epoch limit	25 with early stopping (validation patience about 5)
Augmentation	Training-only time shift and light gain change; avoid label-altering transforms
Selection	Save checkpoint with highest validation macro F1
Reproducibility	Record seed, package versions, label map and spectrogram config
Prediction aggregation	Run deterministic segments; average per-segment probabilities per song

Files and interface

- scripts/05_train_cnn.py reads the manifest and cached log-mels, trains the network and writes genre_cnn.pt.
- The checkpoint contains model_state_dict, class_names, sample_rate, preprocessing parameters, normalization statistics and a version string.
- services/genre.py imports precisely the same architecture and transformations as the training code.

> **AVOID FALSE CERTAINTY** Softmax values are model scores, not automatically calibrated probabilities. If the top score is weak or the gap between top two classes is narrow, show "uncertain / multiple influences" in the UI.

10 Optional mood and instrument inference

The original CNN handles the mandatory research problem: **genre**. Additional mood/instrument tags can come from a published pretrained model stack; they do not require downloading MTG-Jamendo audio or retraining it. This layer should be timeboxed because installation can be platform-sensitive.

Download these six exact model files

Model file	Use
discogs-effnet-bs64-1.pb + .json	Shared audio embedding extractor, ~18 MB weights
mtg_jamendo_moodtheme-discogs-effnet-1.pb + .json	Mood/theme prediction head, ~2.7 MB weights
mtg_jamendo_instrument-discogs-effnet-1.pb + .json	Instrument tagging head, ~2.7 MB weights

Embedding weights directory [S2] https://essentia.upf.edu/models/feature-extractors/discogs-effnet/
Mood model weights + metadata directory [S2] https://essentia.upf.edu/models/classification-heads/mtg_jamendo_moodtheme/
Instrument model weights + metadata directory [S2] https://essentia.upf.edu/models/classification-heads/mtg_jamendo_instrument/

Implementation route

- Run a one-file proof of concept in a compatible Linux/WSL2 environment before wiring the web app. Essentia's default pip distribution does **not** necessarily expose TensorFlow prediction wrappers. [S5]
- Use Essentia's documented MonoLoader at 16 kHz, TensorflowPredictEffnetDiscogs and TensorflowPredict2D; the embedding model is shared for mood and instruments. [S2]
- Read the corresponding JSON metadata class list for UI names; average temporal outputs and show top tags and raw scores.
- Multi-label heads are independent score outputs; do not normalize scores across tags as though they were mutually exclusive genres.
- If runtime support fails within a limited integration window, keep the pretrained layer disabled and ship the validated genre CNN, acoustic analysis and similarity engine.

DEPLOYMENT CAVEAT Essentia TensorFlow wrappers and old tutorial install commands have compatibility constraints. Do not claim Windows-native inference is proven until a local smoke test passes. An isolated Linux runtime or separately validated ONNX implementation is the sensible fallback.

11 Numerical music analysis: what is real and what is estimated

Feature	How to compute	UI wording
Duration / sample rate	Read decoded PCM and metadata	Duration, sample rate
Tempo	librosa beat tracking	Estimated tempo (BPM)
Signal energy	RMS over frames and overall average	Energy profile / RMS
Waveform	Downsample audio peaks for display	Amplitude over time
Spectrogram	STFT / mel transform	Frequency energy over time
Chroma	Chroma CQT/STFT	Pitch-class activation
Key (optional)	Chroma aggregation + key-template correlation	Estimated key (experimental)
Genre	Original CNN	Predicted genre + model scores
Mood / instruments	Optional pretrained classifier	Suggested tags

Neither a chromagram nor pitch detection determines a perfectly reliable musical key. BPM can also produce half-time or double-time estimates. Avoid claims that these quantities are ground truth for every music style.

JSON feature normalization and visualization

- Generate waveform points on the backend using an upper cap (for example, 1,000 values), not full PCM arrays.
- Generate a modest 2D spectrogram matrix (for example, max 128 frequency bins x 300 time bins) or a compressed PNG; do not return tens of megabytes of floats.
- Return timestamps, units, axis labels and min/max scales with every plotted series; distinguish dB from linear magnitude.
- Preserve deterministic preprocessing for the CNN separately from any lightweight plotting downsampling.

12 Song similarity engine

Create a small local content-based retrieval index using only approved FMA tracks and their extracted features. There is no need for Spotify APIs, lyrics, embedding databases or user accounts.

Steps

- Select a fixed catalog of roughly 1,000 to 3,000 FMA tracks from training/validation. Store track IDs, artist, title, genre and non-sensitive metadata.
- Build a numerical vector (MFCC summary, chroma, spectral metrics, RMS, tempo). Fit scaling on the indexed catalog only; persist the scaler and feature order.
- Compute normalized vectors and rank by cosine similarity. Filter identical track hash or track_id to prevent self-retrieval.
- Return the five most similar catalog entries, similarity score and why they appear similar (e.g., tempo, timbre or chroma).
- Show songs as metadata rather than distributing playable files unless the individual Creative Commons license permits redistribution.

Evaluation plan for retrieval

Because exact semantic similarity has no direct labels in the FMA Small dataset, use a transparent proxy evaluation: retrieve nearest neighbors for held-out queries and report same-genre precision@5 as a rough diagnostic, plus qualitative examples. Do not claim it is a user-study measure of musical similarity.

13 Backend: FastAPI endpoint contract

API routes

Endpoint	Contract
GET /api/health	200 JSON: status, model loaded, version, optional tagger availability
POST /api/analyze	multipart/form-data field file; validate MP3, run analysis and return results
GET /api/models	Return trained genres, preprocessing profile and optional model statuses
GET /api/metrics	Read-only academic summary of test metrics for the dashboard
GET /api/catalog/{id}	Return approved display metadata for similar-track details

Example analysis response (illustrative schema)

```
{
  "analysis_id": "local-random-uuid",
  "duration_seconds": 30.0,
  "tempo_bpm_estimate": 121.5,
  "rms_mean": 0.083,
  "estimated_key": null,
  "genre": {
    "model": "sonicscope-cnn-v1",
    "top_label": "Electronic",
    "scores": [{"label": "Electronic", "score": 0.64}]
  },
  "moods": {"status": "disabled", "tags": []},
  "instruments": {"status": "disabled", "tags": []},
  "visualizations": {"waveform": [], "log_mel_db": []},
  "similar_tracks": [],
  "warnings": ["Genre values are model scores, not ground truth."]
}
```

Processing contract

- Use unique temporary paths; never use the uploaded client filename as a filesystem path. Discard files on request completion by default.
- Cap input size (example: 30 MB) and decoded duration (example: 5 minutes). Return clear 4xx error codes for invalid/corrupt audio.
- Decode once when possible; bound CPU/RAM usage; apply deterministic 10-second inference crops over up to the first 30 seconds for v1.
- Return an explicit model status when optional taggers are unavailable. No placeholder random scores should ever appear.
- Start with synchronous local inference. Add a background job queue only if performance measurements justify it.

14 Frontend product specification

Pages and components

Page / component	Function
/ - Upload	Simple drag-and-drop/upload, type and size guidance, MP3 player
/analysis - Results	Overview metrics, top genres, waveform, mel-spectrogram, chroma, energy
/analysis - Optional panels	Mood/instrument tag cards; explain source model and uncertainty
/analysis - Similar tracks	Five catalog neighbors, score and brief comparison explanation
/about - Model card	Dataset provenance, class names, model settings, limitations
/evaluation - Metrics	Macro F1, accuracy, confusion matrix, baseline comparison

UI behavior and visual system

- Use a restrained light theme, spacious grid, high-contrast labels, one accent color and simple waveform-centric iconography.
- Keep primary upload and Analyze actions visible. Loading should show separate stages: decoding, extracting features, running models, preparing charts.
- On the result page, lead with genre, BPM, duration and the waveform. Keep research metrics on a separate evaluation tab.
- Display confidence guidance and model source next to predictions. Do not show invented percent accuracy for individual user uploads.
- On mobile, stack cards; make spectrogram horizontally scrollable or use responsive canvas plots with clear axis tooltips.

Frontend install and local start

```
# From project root in a separate terminal:
npx create-next-app@latest web --ts --tailwind --eslint --app --src-dir
cd web
npm install recharts lucide-react
npm run dev
# Open http://localhost:3000
```

Set NEXT_PUBLIC_API_BASE_URL=http://localhost:8000 in web/.env.local and configure FastAPI CORS for the intended local frontend origin only.

15 Execution flow and integration order

Backend start command

```
# Activate project virtual environment at repository root.
cd backend
python -m uvicorn app.main:app --reload --port 8000
# Open http://localhost:8000/docs for interactive API testing
```

Build in this dependency order

Gate	Only proceed when...
G1: file decoder	Backend can decode 5 representative audio files without exceptions
G2: manifest	Every split is consistent; class label map saved
G3: baseline	Validation predictions and metrics are produced from cached features
G4: CNN	Saved checkpoint can classify an unseen MP3 in a standalone Python CLI
G5: analysis API	POST /api/analyze returns typed JSON for a valid file and rejects bad files
G6: interface	Upload-to-charts runs end to end using actual inference, not mocked results
G7: extensions	Similarity works; optional tagger either works or is clearly disabled
G8: evaluation	Final held-out results exported; demo works without internet

INTEGRATION DESIGN Keep the genre CNN service and acoustic feature service independent. A failure to initialize an optional pretrained instrument model must never crash the central upload-to-genre workflow.

16 Evaluation protocol and experiment matrix

Mandatory results

Experiment	Metrics and required visuals
Random Forest baseline	Held-out accuracy, macro F1, per-class F1 and confusion matrix
CNN, best checkpoint	Same track-level metrics and confusion matrix
Model comparison	Compact table: test accuracy, macro F1, inference latency
Genre errors	3-5 misclassified examples and discussion of overlapping genres
Data quality	Class counts, exclusions and per-split totals
Similarity	Qualitative top-5 lists; optional same-genre precision@5 proxy
Performance	Decode time, feature extraction time, CNN inference time on named hardware

Evaluation safeguards

- Preserve official train / validation / test song assignments. Report sample counts **after** excluding corrupt entries.
- Train and tune only on training/validation; run final test after the checkpoint and thresholds are frozen.
- Average segment outputs to track level before computing final track-level metrics.
- Plot a normalized and raw-count confusion matrix; include eight original FMA genre names, not a custom handpicked genre taxonomy.
- Record actual experiment parameters and results in reports/metrics/ and do not fabricate target accuracy.

Suggested experiment summary table

Model	Input	Accuracy	Macro F1
Random Forest	Handcrafted summary vector	TBD after testing	TBD
2D CNN	10-second log-mel	TBD after testing	TBD

17 Security, reliability and test checklist

Test case	Expected result
Valid MP3: 15 s, 30 s	Analysis succeeds; padding/cropping handled as defined
Very short or silent file	Graceful validation or an explicit low-signal warning
Non-MP3 renamed to .mp3	Reject on actual decode / signature check
Corrupt file / oversized file	4xx response with useful message, not a server crash
Duplicate upload	Deterministic result within numerical tolerances
Model checkpoint missing	Health endpoint reports unavailable; understandable error
Tagging integration disabled	Genre and visualizations still work normally
Similarity index missing	No fake suggestions; show feature temporarily unavailable
Frontend API disconnected	Clear retry behavior; never display simulated outputs
Slow CPU	User sees progress and no repeated accidental submissions

Privacy and compliance

- Uploaded user files remain local where possible and are deleted after processing; no login or tracking is required.
- Do not expose training data binaries publicly. Respect each FMA track's licensing metadata and MTG pretrained-model non-commercial restrictions. [S1-S3]
- Use a practical local-only deployment for evaluation; if deployed publicly, add stricter upload limits, access controls, concurrent-job caps and security testing.

18 14-day implementation schedule

Days	Concrete engineering tasks	Acceptance output
1-2	Install tools; download and verify ZIPs; create repo	Folder tree + verified dataset checksums
3	Read metadata; manifest; data audit and exclusions	manifest.csv with fixed splits and labels
4	Decode, plot features; cache training log-mels	EDA plots + 5 sample spectrograms
5	Extract baseline feature vectors; train Random Forest	Baseline checkpoint + validation metrics
6-7	Implement CNN; train; early stop; save best model	Working genre_cnn.pt and log plots
8	Standalone upload inference + core FastAPI API	Working /api/analyze from Swagger
9-10	Next.js upload and chart dashboard	Real upload-to-analysis interface
11	Build catalog similarity and metadata views	Five related track results
12	Optional Essentia compatibility spike and integration	Real tags or explicit disabled status
13	Final test evaluation, bug fixes and screenshots	Test metrics, error cases and figures
14	Report, slides, demo rehearsal and packaging	Reproducible submission bundle

If the schedule slips, preserve this order

P0: training data, original CNN, upload API, core plots, evaluation. **P1:** Random Forest comparison, similarity index. **P2:** pretrained mood/instruments, extra exports. If core training runs slowly, cache log-mels and temporarily use fewer epochs/limited training songs, but disclose pilot experiments separately from final results.

19 Academic report and presentation deliverables

Final project report structure

- 1. Abstract, introduction, problem statement and objectives
- 2. Literature overview: music information retrieval, spectrograms, CNNs, feature engineering
- 3. Dataset provenance, genre distribution, original splits, licenses and preprocessing
- 4. Proposed architecture, data flow, ML algorithms and tools
- 5. Implementation: ingestion, baseline, CNN, analysis API, frontend and similarity
- 6. Experimental setup, evaluation protocol, metrics and error analysis
- 7. UI screenshots, observed limitations, future improvements, conclusion and references

Eight-slide evaluation presentation

1) Problem and product; 2) Dataset and input pipeline; 3) Spectrogram visual data; 4) Baseline vs CNN architecture; 5) System architecture and UI; 6) Measured results; 7) Live upload demo; 8) Limitations and takeaway.

Final submission packaging

File / folder	Must contain
README.md	Prerequisites, setup, dataset sources, exact commands, what is optional
scripts/	Manifest builder, processing, training, evaluation, similarity setup
backend/ and web/	Working source code with .env.example and documented configuration
models/checkpoints/ (local)	Trained checkpoint with label mapping; distribute if permitted by license/policy
reports/	Report PDF, metric JSONs, confusion matrix, UI screenshots and architecture
demo/ (optional)	Short offline screen recording; only use audio with demo rights

LIVE DEMO SCRIPT Upload one known FMA test song and one external legally held MP3. Show the genre ranking, open the spectrogram and waveform, explain the CNN input, then open the evaluation tab to compare against Random Forest. Avoid claiming the external MP3 has a verified label unless you actually know it.

20 Risks, fallback decisions and non-goals

Risk	Mitigation
Large download interrupted	Use restart/resume-capable download; verify checksums before unzip
Corrupt FMA audio or metadata quirks	Decode audit, stable exclusion log, official FMA errata
No dedicated GPU	Small CNN, 16/32 batch, mixed cache and training patience; CPU possible but slower
Validation score disappointing	Document actual results; review splits, normalization and model underfit
Genre ambiguous for real-world upload	Show top 3 scores and uncertainty note; do not assert absolute genre
Essentia TF wrappers unavailable	Stop after timeboxed spike; mark optional tags disabled
Slow analysis response	Analyze at most first 30s for v1; precompute visualization samples
Music redistribution restrictions	Display approved metadata; do not embed raw public audio catalog

Explicitly excluded from v1

Real-time microphone streaming, copyrighted song recognition, exact artist identification, lyrics generation/transcription, stem separation, music generation, full-length music streaming, large vector databases, personalized accounts and a full production recommendation service.

21 Primary references and direct model links

The following are publisher or official project references consulted for this implementation plan. Access and package compatibility should still be rechecked at installation time. All links are clickable.

[S1] FMA official GitHub README, data description and checksums https://github.com/mdeff/fma
[S1] FMA paper https://arxiv.org/abs/1612.01840

[S2] Essentia model catalog and example prediction code https://essentia.upf.edu/models.html
[S3] MTG-Jamendo official repository and licensing https://github.com/MTG/mtg-jamendo-dataset
[S4] Librosa documentation https://librosa.org/doc/latest/index.html
[S5] Essentia model inference / machine learning guidance https://essentia.upf.edu/machine_learning.html
Direct model: Discogs-EffNet shared encoder (.pb) https://essentia.upf.edu/models/feature-extractors/discogs-effnet/discogs-effnet-bs64-1.pb
Direct model: Mood head (.pb) https://essentia.upf.edu/models/classification-heads/mtg_jamendo_moodtheme/mtg_jamendo_moodtheme-discogs-effnet-1.pb
Direct model: Instrument head (.pb) https://essentia.upf.edu/models/classification-heads/mtg_jamendo_instrument/mtg_jamendo_instrument-discogs-effnet-1.pb

Direct metadata files to accompany each model

Shared encoder metadata (.json) https://essentia.upf.edu/models/feature-extractors/discogs-effnet/discogs-effnet-bs64-1.json
Mood labels and metadata (.json) https://essentia.upf.edu/models/classification-heads/mtg_jamendo_moodtheme/mtg_jamendo_moodtheme-discogs-effnet-1.json
Instrument labels and metadata (.json) https://essentia.upf.edu/models/classification-heads/mtg_jamendo_instrument/mtg_jamendo_instrument-discogs-effnet-1.json
The optional pretrained models are primarily intended for non-commercial research and have licensing restrictions. This document is an academic implementation plan, not a permission grant to redistribute audio or deploy commercial model services.

22 Final step-by-step execution checklist

- Install Git, Python 3.11, Node.js and FFmpeg; verify versions.
- Create project folder; initialize Git and Python virtual environment.
- Download fma_small.zip and fma_metadata.zip; verify both SHA-1 checksums.
- Extract to data/raw/; inspect tracks.csv and five decoded MP3s.
- Build official-split manifest.csv; audit file count and class labels.
- Create feature vectors and log-mel spectrogram cache.
- Train and validate Random Forest baseline; save its fitted pipeline.
- Train and validate 2D CNN; save the best checkpoint and label map.
- Implement single-file inference; compare outputs across repeated runs.
- Build FastAPI POST /api/analyze and health endpoint; test bad files.
- Create Next.js MP3 upload page, charts and ranked genre cards.
- Build five-nearest-tracks similarity index and catalog UI.
- Optionally run pretrained mood/instrument integration spike.
- Run final held-out evaluation once; export tables and plots.
- Finish report, README, slides, offline demo and QA checklist.

MINIMUM DELIVERABLE When time is short, stop after a working upload-to-genre prediction, real spectrogram/chroma charts and a documented CNN-vs-baseline experiment. Those are the core academic outcomes. Everything else improves polish but should never block submission.